

EVENTFLOW

EF-DOC-001

Vision & Design Principles

Version 0.2 | Draft for Product Owner Review

Every Guest Matters. Every Event Flows.

Document Control

Document ID	EF-DOC-001
Title	EventFlow Vision & Design Principles
Version	0.2
Status	Draft for Product Owner Review
Product	EventFlow
Document Owner	Product Owner
Technical Owner	Solution Architect
Created	2026-08-06
Last Updated	2026-08-06
Review Frequency	Major product or architectural change
Classification	Internal Product Strategy & Design Governance
Related Documents	EF-DOC-000, EF-DOC-002, EF-DOC-003, EF-DOC-004

Revision History

Version	Date	Status	Summary
0.1	2026-08-06	Draft	Initial vision and principles captured during product formation.
0.2	2026-08-06	Draft for Review	Formalized product vision, operating principles, quality attributes, capacity philosophy, module boundaries, user experience standards, and growth model.

Approval Record

Role	Approval	Date
Product Owner	Pending	-
Solution Architect	Prepared	2026-08-06

1. Purpose

This document defines the strategic product vision and design principles that guide EventFlow. It translates the EventFlow Constitution into practical product and engineering direction without prescribing low-level implementation details.

It is intended to answer four questions consistently throughout the product lifecycle:

1. What is EventFlow?
2. Who is EventFlow for?
3. What qualities must EventFlow preserve as it grows?
4. What principles should guide trade-offs when requirements compete?

Where this document conflicts with EF-DOC-000, the EventFlow Constitution prevails.

2. Product Vision

EventFlow is a professional Event Operations Platform that enables organizers to manage the complete guest lifecycle - from event configuration and invitation through communications, attendance confirmation, seating, reception, check-in, and post-event reporting - within one coherent system.

The long-term vision is a platform that can support private celebrations, weddings, church functions, conferences, fundraisers, corporate events, and other organized gatherings without requiring event-specific source-code changes.

3. Mission

EventFlow exists to reduce operational friction in event management by combining:

- structured guest and invitation data;
- simple guest-facing workflows;
- reliable communications;
- operational tools for organizers and event staff;
- controlled automation;
- trustworthy reporting;
- reusable event configuration.

The product should make complex event logistics feel manageable without hiding important operational state from organizers.

4. Product Positioning

EventFlow shall be positioned as an **Event Operations Platform**, not merely as:

- an RSVP form;
- an invitation designer;
- a mailing tool;
- a seating planner;
- a check-in application;
- a WordPress form plugin.

These may be capabilities within EventFlow, but the product value comes from their integration around a shared event and attendee model.

5. Initial Reference Implementation

The first live implementation is the **Lui @ 60** celebration.

This implementation serves as:

- the first production proving ground;
- a source of real operational requirements;
- a usability test environment;
- a validation case for invitation, companion, communication, seating, and reception workflows.

EventFlow must learn from this event without becoming coupled to birthdays, a specific venue, a specific invitation design, or a single event structure.

6. Target Users

6.1 Product Administrator

Responsible for system configuration, integrations, global permissions, updates, and platform health.

6.2 Event Organizer

Owns an event and manages invitation strategy, guest data, communications, seating, event settings, and reports.

6.3 Event Coordinator

Supports day-to-day operations including guest records, companion details, communication follow-up, table preparation, and event logistics.

6.4 Reception Staff

Requires a deliberately limited interface optimized for finding attendees, confirming table assignments, and recording arrival.

6.5 Guest / Invitee

Receives a secure event invitation and performs only the actions required by that invitation, such as reviewing event details and confirming attendee information.

7. Product Principles

DP-001 - Event First

The Event is the primary organizing context for EventFlow. Product navigation, permissions, data ownership, reports, and integrations should make event context explicit.

DP-002 - Invitation and Attendee Are Distinct

An Invitation represents the organizer's invitation relationship and capacity entitlement. An Attendee represents a person.

This separation enables:

- couples and families on one invitation;
- group invitations;
- individual seating;
- individual check-in;
- individual badge or place-card generation;
- invitation-level communications;
- attendee-level operations.

DP-003 - Configuration Over Hard-Coding

Ordinary event variation must be represented through settings or structured data.

A new event should not require edits to source code to change its:

- name;
- venue;
- schedule;
- invitation image;
- wording;
- theme;
- email templates;
- guest limits;
- deadlines.

DP-004 - Single Source of Truth

Each business fact should have an authoritative owner.

Examples:

- invitation capacity belongs to the Invitation;
- a person's identity belongs to the Attendee;
- table capacity belongs to the Table;
- delivery status belongs to the Communication record;
- event date belongs to Event configuration.

Dashboards and reports derive from these sources rather than maintaining competing copies.

DP-005 - Mobile-First Guest Experience

Guest-facing interfaces must assume a smartphone as the primary device.

The guest should not need to:

- zoom the page;
- understand WordPress;
- create an account unless genuinely necessary;
- copy long codes manually;
- navigate unrelated website content.

DP-006 - Operational Clarity Over Decorative Complexity

Visual design should reinforce the event's branding while preserving clear status, actions, and error messages.

An attractive page that obscures what the guest or organizer must do is a failed design.

DP-007 - Progressive Disclosure

Users should see the information and controls appropriate to their current task.

Examples:

- reception staff should not see API credentials;
- guests should not see administrative data;
- organizers should not be overwhelmed with advanced settings during basic event setup.

DP-008 - Safe Defaults

The default behavior should minimize accidental data exposure, unintended bulk sends, duplicate communications, oversubscription, and destructive administrative actions.

DP-009 - Audit Important Actions

Material changes must be traceable where practical, including actor, time, event, action, and affected object.

DP-010 - External Services Are Replaceable Dependencies

Email, SMS, payments, maps, file storage, and other third-party providers are integrations, not the EventFlow domain model.

Core event data should remain usable if one provider is replaced or temporarily unavailable.

DP-011 - Build for Recovery

Critical operations should be recoverable.

Examples include:

- re-running an interrupted import without duplication;
- retrying failed messages;
- restoring data from backup;
- reversing a mistaken table assignment;
- correcting attendee information without destroying history.

DP-012 - Scale by Design, Not by Assumption

The data model and core workflows should not require redesign simply because an event grows from hundreds to thousands of attendees.

Scaling mechanisms may evolve, but core domain relationships should remain valid.

8. Core Quality Attributes

EventFlow's architecture and requirements shall explicitly address the following attributes.

8.1 Usability

The product should minimize training requirements for occasional users and guests.

8.2 Reliability

Event-day workflows must remain stable and predictable under operational pressure.

8.3 Performance

Common guest and staff actions should feel immediate at normal event scale. Large operations should use batching or asynchronous processing when appropriate.

8.4 Scalability

The initial architecture should comfortably support small events and be capable of evolving to support events with several thousand attendees without fundamental redesign.

8.5 Security

Access control, token security, secure integration credentials, validation, escaping, and auditability are baseline requirements.

8.6 Maintainability

The platform should use clear module boundaries, documented interfaces, controlled migrations, and readable code.

8.7 Accessibility

Public experiences should align with WCAG-informed design and semantic web practices.

8.8 Portability

EventFlow should avoid unnecessary coupling to a single hosting provider or external service where reasonable.

8.9 Observability

As the platform matures, operators should be able to determine whether imports, message sends, check-ins, integrations, and background operations are healthy.

9. Capacity Philosophy

Capacity must be expressed as measurable targets in the PRD and Architecture documents. This document establishes the direction, not final service-level commitments.

9.1 Reference Capacity Tiers

Tier A - Small Event

- Up to 100 attendees.
- Single organizer.
- Low communication volume.

Tier B - Standard Event

- 100 to 750 attendees.
- Multiple staff roles.
- Bulk email/SMS.
- Table planning and event-day check-in.

Tier C - Large Event

- 750 to 5,000 attendees.
- Multiple coordinators and reception staff.
- Queued bulk communications.
- Indexed and paginated administration.
- Background imports/exports and reporting.

Tier D - Future Platform Scale

- More than 5,000 attendees and/or many concurrent events.
- Requires explicit infrastructure, queueing, caching, observability, tenancy, and service-capacity design.

EventFlow v1.x does not promise Tier D production capacity, but the architecture should avoid choices that make Tier D impossible without replacing the domain model.

10. Core Product Modules

10.1 Events

Creates and configures the event context, schedule, venue, branding, lifecycle status, and event-wide rules.

10.2 Invitations

Manages invitees, secure access, allowed capacity, invitation status, and guest-facing invitation experience.

10.3 Attendees

Represents actual individuals associated with an event, whether primary guests or companions.

10.4 Communications

Manages templates, campaigns, delivery attempts, channel providers, reminders, and message history.

10.5 Seating

Manages tables, table capacities, assignments, grouping constraints, and later visual layout capabilities.

10.6 Reception & Check-in

Provides fast attendee search, table lookup, arrival recording, and operational event-day views.

10.7 Reports & Analytics

Provides invitation, attendance, communications, seating, check-in, and operational reporting.

10.8 Administration

Manages users, roles, permissions, system configuration, integrations, audit records, health information, and controlled maintenance functions.

11. Module Boundary Principles

1. Modules own their domain rules.
2. Cross-module operations use defined services or contracts.
3. User interfaces do not become the source of business rules.
4. Reports generally consume state rather than mutate it.
5. Integration adapters translate external provider behavior into EventFlow's internal model.
6. Shared concepts are defined once.
7. Database access should not become an uncontrolled substitute for module interfaces as the product matures.

12. Guest Experience Principles

A guest-facing journey should:

1. open from a secure personalized link;

2. clearly identify the event;
3. display the event's official branding and invitation information;
4. explain what action is required;
5. show invitation capacity where relevant;
6. collect only information necessary for the current stage;
7. confirm successful submission clearly;
8. permit organizer-controlled correction or reopening when required;
9. avoid disclosing other guests or internal administrative information.

13. Organizer Experience Principles

Organizer workflows should favor:

- bulk actions with review before execution;
- filters and saved operational views;
- visible counts and statuses;
- data validation before import or send;
- reversible operations where practical;
- clear separation between draft, queued, sent, failed, and completed states;
- exportability of important event data;
- explicit warnings before destructive actions.

14. Event-Day Experience Principles

Reception and event-day interfaces are operational tools.

They should prioritize:

- speed;
- large touch targets;
- minimal navigation;
- search by primary guest or companion;
- immediate table information;
- clear check-in state;
- protection against accidental duplicate actions;
- resilience when non-critical external services are unavailable.

15. Integration Principles

1. Provider-specific code should be isolated behind integration adapters.

2. External provider identifiers should not replace EventFlow's internal identifiers.
3. Integration credentials must not be committed to source control.
4. Delivery attempts should retain EventFlow-side history independent of provider dashboards.
5. Webhooks, where used, must be authenticated or verified according to provider capabilities.
6. A failed provider should not corrupt event data.
7. Integration replacement should not require restructuring Events, Invitations, or Attendees.

16. Data Principles

EventFlow data should be:

- identifiable through stable internal IDs;
- event-scoped;
- validated at boundaries;
- timestamped where lifecycle history matters;
- migratable;
- exportable where operationally important;
- auditable for material changes;
- minimized to information required for legitimate event operations.

Personally identifiable information should not be added merely because it might be useful later.

17. Growth Model

Stage 1 - Reference Implementation

Single EventFlow deployment supporting Lui @ 60 and proving the core guest lifecycle.

Stage 2 - Reusable Single-Event Product

EventFlow can be configured for a different event without code modification.

Stage 3 - Multi-Event Platform

One installation can manage multiple events with explicit event scoping and permissions.

Stage 4 - Organization-Aware Platform

Organizations manage multiple events, users, roles, branding, and shared resources.

Stage 5 - SaaS / Hosted Product

EventFlow becomes a hosted multi-tenant platform with account provisioning, subscription capabilities, operational observability, stronger tenancy isolation, and service-level capacity management.

Each stage requires formal requirements and ADR review before implementation.

18. Product Decision Test

A proposed capability should normally satisfy the following questions before entering implementation:

1. What user problem does it solve?
2. Which EventFlow module owns it?
3. Is it event-specific configuration or reusable product behavior?
4. Does it change an existing ADR?
5. Does it introduce new personal data?
6. What happens when the feature or external provider fails?
7. How does it behave at Tier A, Tier B, and Tier C scale?
8. What acceptance criteria prove that it works?
9. Which controlled documents must be updated?
10. Does it preserve the EventFlow Constitution?

19. Non-Goals for Early Releases

Early EventFlow releases do not need to solve every aspect of event management.

Unless approved through the PRD, early versions shall not expand into:

- accounting;
- vendor procurement;
- hotel booking;
- travel management;
- full CRM functionality;
- general-purpose marketing automation;
- unrelated website content management.

These capabilities may integrate with EventFlow later, but they should not dilute the core event operations domain.

20. Success Definition

EventFlow succeeds when:

- an organizer can configure and understand an event without technical assistance;
- guests can complete their required invitation journey without support;
- staff can operate reception efficiently;
- event data remains coherent across invitations, attendees, communications, seating, and check-in;
- the software remains understandable and maintainable as features increase;
- new event types are created through configuration rather than source-code forks;
- documentation and implementation remain traceable to requirements and decisions.

21. Guiding Statement

EventFlow should make the operational complexity of an event visible to the organizer, but simple for the guest.

This principle complements the product motto:

Every Guest Matters. Every Event Flows.